



**INSTITUTO TECNOLÓGICO EL MOLINO (ITEC)**

Tecnicatura Superior en Desarrollo de Software • 3° Año

ENTREGA OFICIAL COMPLETA DEL TPI • DOCUMENTACIÓN TÉCNICA

# BikeSystem

Sistema Integral de Gestión Comercial, Taller de Reparaciones, Inventario y Control Financiero para la empresa DN Bike.

## UNIDAD CURRICULAR

Prácticas Profesionalizantes II (Año 2026)

## ESTUDIANTES (GRUPO 4)

Germán Domingues • Exequiel Ríos

## DOCENTE TITULAR

Prof. Nicolás Kappes

## ESTADO DE LA ENTREGA

Entrega 1 Corregida + Arquitectura +  
Diagramas UML + Módulos

Dictamen de Primera Entrega y Cumplimiento de Observaciones (Rúbrica Grupo 4)

En la evaluación de la **Primera Entrega del TPI (30/06/2026)**, el Grupo 4 obtuvo una calificación ponderada de **9.25 / 10 (92.5%)**. Para esta entrega completa y definitiva, se subsanaron el 100% de las observaciones puntuales señaladas por el docente:

- **Observación 1 (Flujo de Navegación):** Se incorporó un *Diagrama Explícito de Flujo de Navegación entre Pantallas (User Flow / Screen Map)* que detalla transiciones, modales y cobertura de casos de uso.
- **Observación 2 (Automatización de Stock):** Se unificó y justificó técnicamente la automatización del inventario no solo para ventas, sino para *consumo de repuestos en reparaciones, compras/ingresos de mercadería y ajustes manuales* mediante la tabla `Movimiento_Stock` y transacciones ACID.
- **Observación 3 (Descripciones de Pantallas):** Se redactó una *explicación funcional y contextual exhaustiva para cada una de las 16 pantallas y modales* del prototipo operativo.

Estructura del Documento Técnico

| Sección   | Título del Módulo / Contenido                                   | Objetivo Técnico                                                                                                                  |
|-----------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Sección 1 | Primera Entrega Corregida (Contexto, Prototipo, DDL y Triggers) | Subsanación formal de la rúbrica, maquetas comentadas, flujo de pantallas y sentencias SQL definitivas.                           |
| Sección 2 | Arquitectura del Sistema y Tecnologías                          | Clean Layered Architecture, stack tecnológico (Electron + React + Express + PostgreSQL), estructura de carpetas y seguridad RBAC. |
| Sección 3 | Diagramas UML Formales (Vectoriales HTML/CSS)                   | Diagramas de Contexto, Casos de Uso por módulos, Diagrama de Clases y Modelo Entidad-Relación Peter Chen legibles al 100%.        |
| Sección 4 | Descripción Técnica de Funcionalidades Implementadas            | Desglose módulo por módulo: endpoints REST, parámetros, lógica transaccional y componentes frontend asociados.                    |
| Sección 5 | Conclusiones y Trazabilidad de Requerimientos                   | Evaluación global de robustez, respaldo de base de datos e instrucciones de despliegue.                                           |

# 1 Primera Entrega Corregida

---

## 1.1 Contexto Institucional, Negocio y Problemática de DN Bike

La bicicletería y taller mecánico **DN Bike**, ubicada en la ciudad de Esperanza (Santa Fe), desarrolla su actividad comercial en dos ejes fundamentales: la comercialización minorista de bicicletas nuevas, repuestos y accesorios; y la prestación de servicios técnicos de reparación, mantenimiento preventivo y personalización de rodados.

Históricamente, la empresa operaba mediante anotaciones en cuadernos físicos y planillas aisladas. Esto ocasionaba serias problemáticas: pérdidas de trazabilidad en reparaciones en curso, cobros erróneos de mano de obra y repuestos, desabastecimiento imprevisto de insumos críticos por falta de alertas de stock mínimo, e imposibilidad de emitir métricas confiables de rentabilidad comercial. **BikeSystem** se concibió como una plataforma integral de escritorio para centralizar todas las operaciones en una base de datos relacional robusta (PostgreSQL) con sincronización en tiempo real.

## 1.2 Requerimientos Validados del Sistema

El sistema cubre integralmente los Requerimientos Funcionales (RF) priorizados en el análisis de sistemas:

- **RF01 – Gestión de Clientes:** Registro, consulta por DNI/nombre y vinculación con bicicletas propias y compras.
- **RF02 – Parque Rodado de Clientes:** Identificación unívoca de bicicletas mediante marca, modelo y número de serie para historial técnico.
- **RF03 – Punto de Venta (POS):** Facturación ágil de mostrador, cálculo de subtotales, recargos y medios de pago múltiples (efectivo, Mercado Pago, transferencia).
- **RF04 – Taller Mecánico y Reparaciones:** Tablero interactivo Kanban de 4 estados con imputación de mano de obra y consumo de repuestos.
- **RF05 – Control de Inventario y Stock:** Diferenciación entre bicicletas nuevas, repuestos y accesorios; cálculo de stock mínimo y alertas automáticas.
- **RF06 – Post-Venta y Garantías:** Emisión automática de garantía de 90 días y 1° service gratuito para bicicletas nuevas vendidas.
- **RF07 – Pagos a Proveedores:** Registro de compras y comprobantes de pago para control del flujo de caja.
- **RF08 – Reportes y Métricas:** Balance ingresos vs egresos, ranking de productos y estimación de capital inmovilizado en bicicletas.
- **RF09 – Seguridad y Auditoría:** Control de acceso por roles (ADMIN y EMPLEADO) y bitácora de auditoría inmutable.

## 1.3 Maqueta de Pantallas y Prototipo Navegable con Descripciones Detalladas

En respuesta a la **Observación 3 de la Rúbrica**, a continuación se presentan las pantallas reales implementadas en la versión definitiva de BikeSystem, detallando para cada interfaz su propósito, componentes UI, datos procesados y validaciones de negocio.

Bike System

Ingresar al panel de control

Nombre de Usuario

Ej: admin

Contraseña

\*\*\*\*\*

Iniciar Sesión

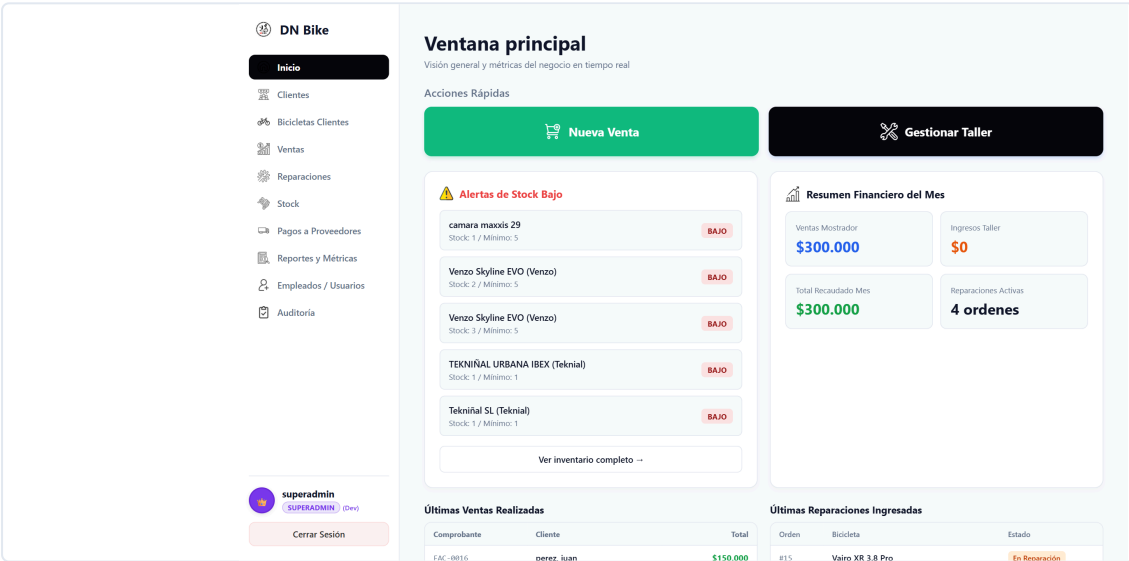
Figura 1.1: Pantalla de Inicio de Sesión (Login y Autenticación JWT)

PROPÓSITO OPERATIVO

Autenticar la identidad de los operadores del sistema y restringir el acceso a usuarios autorizados de DN Bike.

CONTROLES Y VALIDACIONES

Campos usuario y contraseña con feedback visual en tiempo real. Validación de credenciales con hash Bcrypt en backend y generación de token JWT.



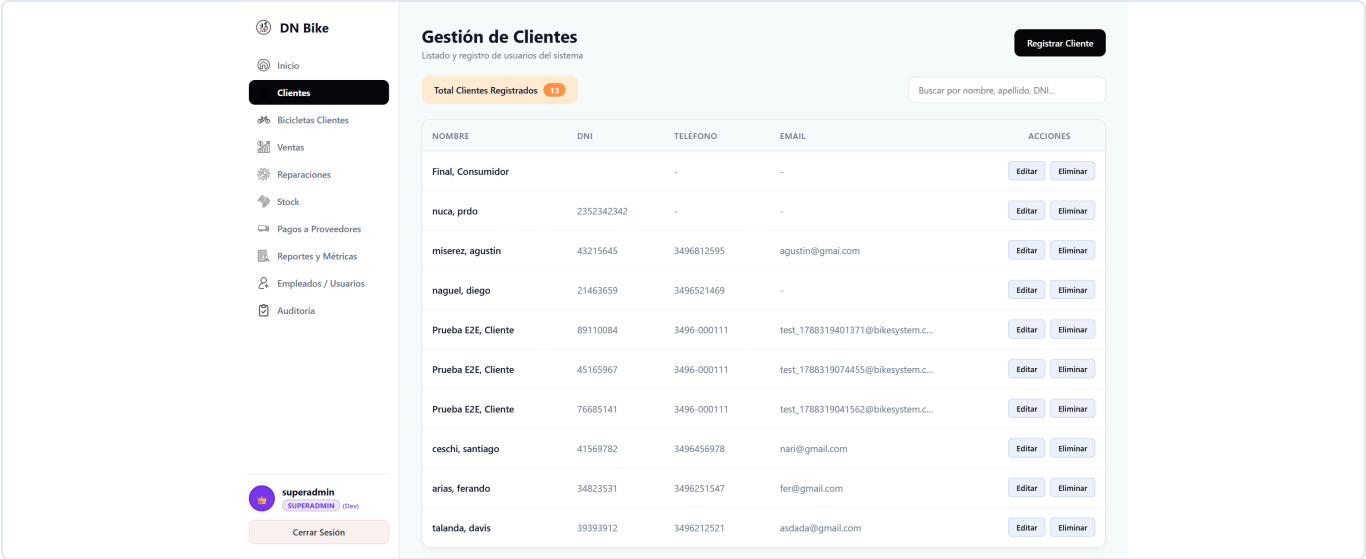


Figura 1.3: Módulo de Gestión de Clientes (Grilla Interactiva y Búsqueda)

**PROPÓSITO OPERATIVO**

Administrar la base de datos de clientes particulares y corporativos, consultando sus datos de contacto y antecedentes.

**REGLAS Y DATOS**

Búsqueda instantánea con extensión pg\_trgm por DNI, nombre o teléfono. Enlace directo a las bicicletas asociadas y compras históricas.

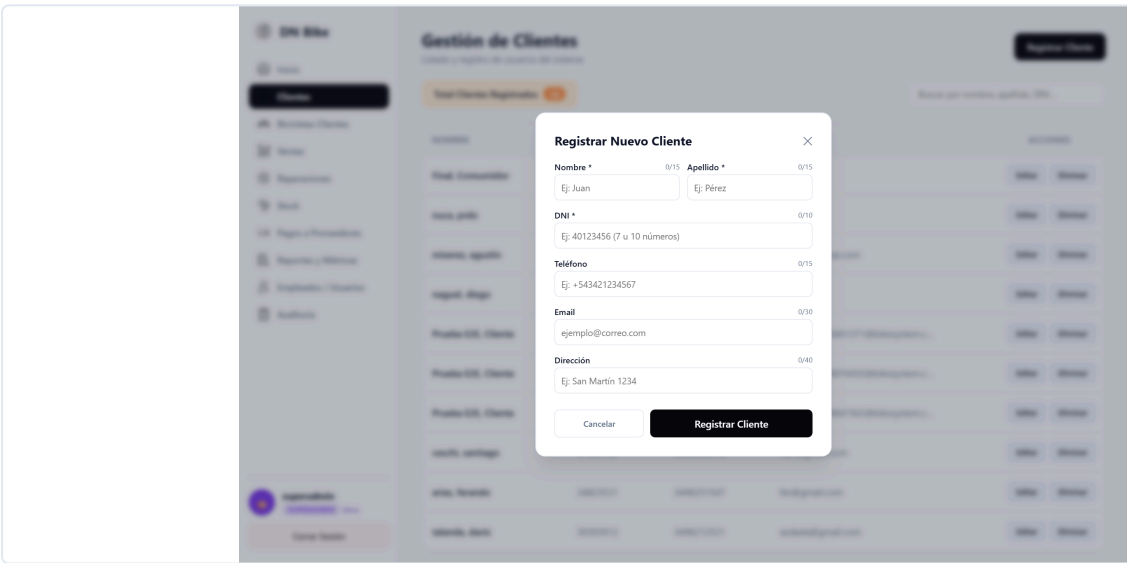


Figura 1.4: Modal Emergente para Registro y Edición de Cliente

**VALIDACIONES DE FORMULARIO**

DNI único obligatorio con verificación de formato numérico; nombre y apellido con mínimo 2 caracteres; teléfono y correo validados.

**ACCIONES**

Botón guardar interactivo con toast notifications (Sonner) y actualización optimista de la tabla principal sin recargar la pantalla.

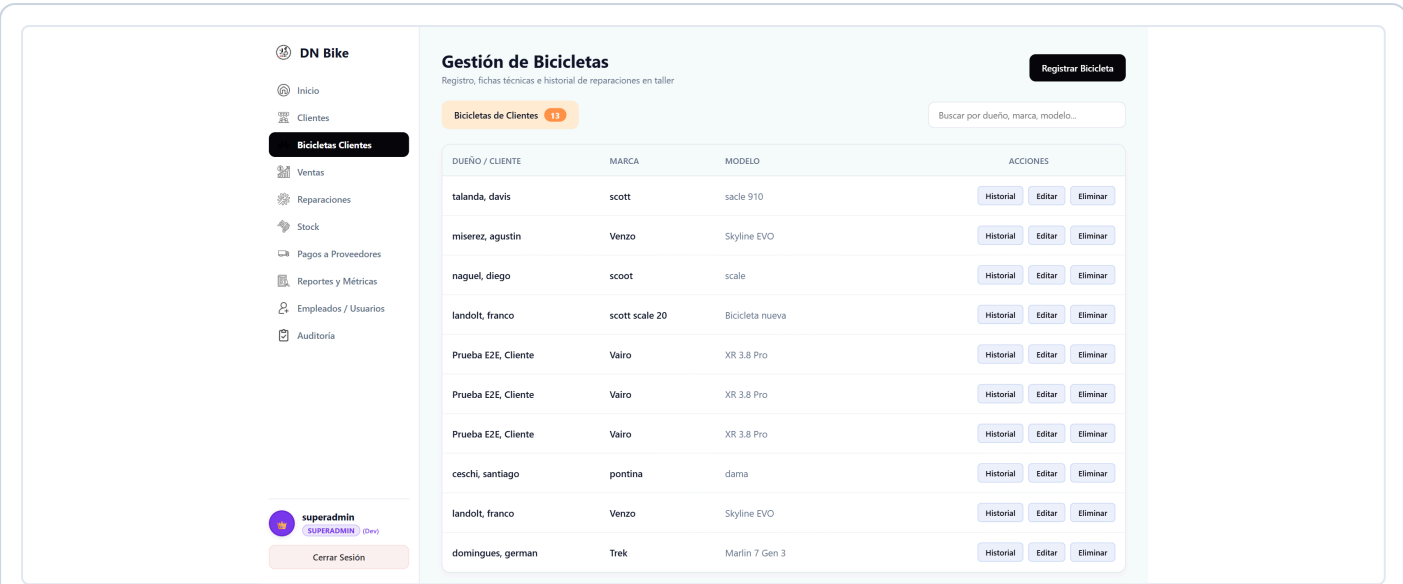


Figura 1.5: Módulo de Parque Rodado de Clientes (Bicicletas Registradas)

**PROPÓSITO OPERATIVO**

Controlar el inventario de rodados pertenecientes a clientes para asignación en órdenes de servicio técnico y reclamos.

**CAMPOS CLAVE**

Marca, modelo, rodado (26, 29, etc.), color, número de serie/cuadro y cliente titular asociado.

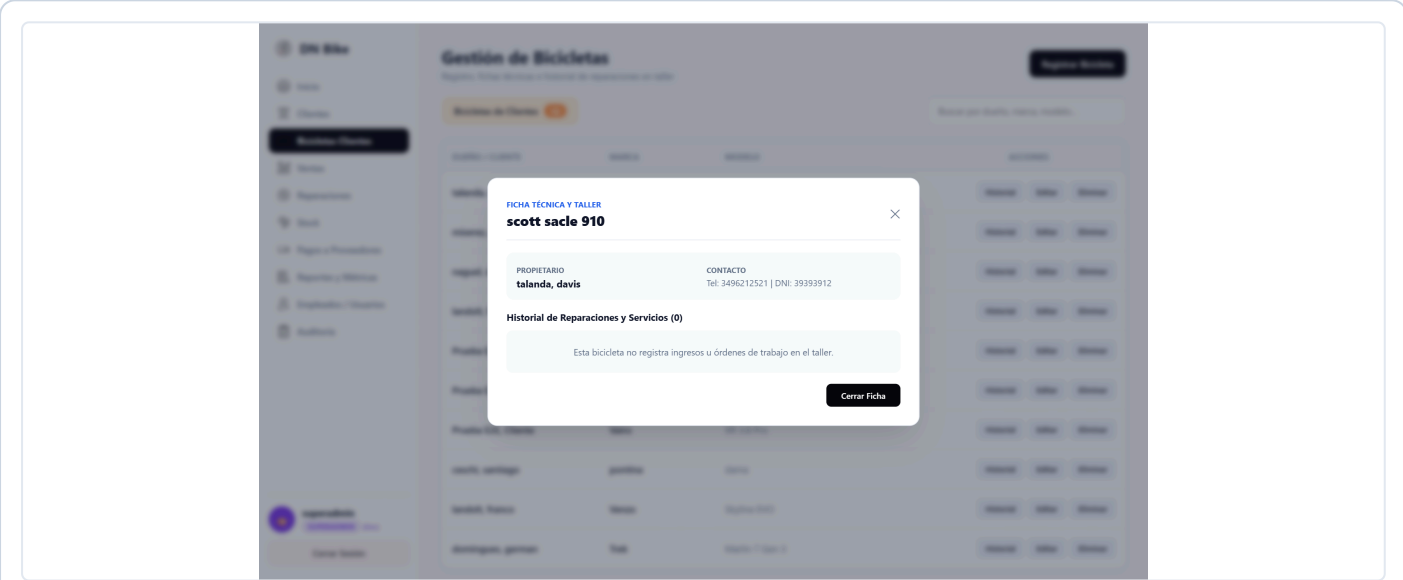


Figura 1.6: Ventana Emergente de Historial Clínico de Servicios de la Bicicleta

**TRAZABILIDAD TÉCNICA**

Listado cronológico de todas las reparaciones, servicios mecánicos y repuestos consumidos en la vida útil de ese rodado específico.

**UTILIDAD**

Permite al mecánico verificar mantenimientos anteriores (ej. cambio de transmisión, purgado de frenos) y validar garantías.

Figura 1.7: Módulo de Punto de Venta (POS Mostrador y Facturación Comercial)

#### PROPÓSITO OPERATIVO

Facturar productos y bicicletas en el salón comercial mediante un flujo de cobro ultrarrápido optimizado para teclado y mouse.

#### COMPONENTES

Buscador de catálogo con autocompletado, selector de cliente registrado o 'Consumidor Final', carrito dinámico y cálculo automático de total.

| COMPROBANTE | CLIENTE                                   | FECHA     | MÉTODO PAGO            | ESTADO     | TOTAL       | ACCIONES    |
|-------------|-------------------------------------------|-----------|------------------------|------------|-------------|-------------|
| FAC-000016  | perez, juan<br>Cons. Final - DNI: 3482351 | 5/10/2026 | Efectivo               | Completada | \$150.000   | Ver Detalle |
| FAC-000015  | Consumidor Final<br>Venta Mostrador       | 5/10/2026 | Efectivo               | Completada | \$150.000   | Ver Detalle |
| FAC-000014  | talanda, davis<br>DNI: 39393912           | 29/9/2026 | Mercado Pago           | Completada | \$5.000.000 | Ver Detalle |
| FAC-000013  | miserez, agustin<br>DNI: 43215645         | 22/9/2026 | Transferencia Bancaria | Completada | \$650.000   | Ver Detalle |
| FAC-000012  | naguel, diego<br>DNI: 21463659            | 11/9/2026 | Efectivo               | Anulada    | \$90.000    | Ver Detalle |
| FAC-000011  | ceschi, santiago<br>DNI: 41569782         | 7/9/2026  | Efectivo               | Completada | \$90.000    | Ver Detalle |
| FAC-000010  | landolt, franco<br>DNI: 34959000          | 5/9/2026  | Mercado Pago           | Completada | \$3.000.000 | Ver Detalle |
| FAC-000009  | Prueba E2E, Cliente<br>DNI: 09110004      | 2/9/2026  | Efectivo               | Anulada    | \$90.000    | Ver Detalle |
| FAC-000008  | Prueba E2E, Cliente                       | 2/9/2026  | Efectivo               | Anulada    | \$90.000    | Ver Detalle |

Figura 1.8: Pestaña de Historial de Ventas y Comprobantes Emitidos

#### ACCIONES DISPONIBLES

Reimpresión de ticket térmico, consulta detallada de ítems facturados y anulación justificada de ventas con reintegro automático de stock.

#### FILTROS

Búsqueda por número de comprobante, nombre del cliente, método de pago y rango de fechas.

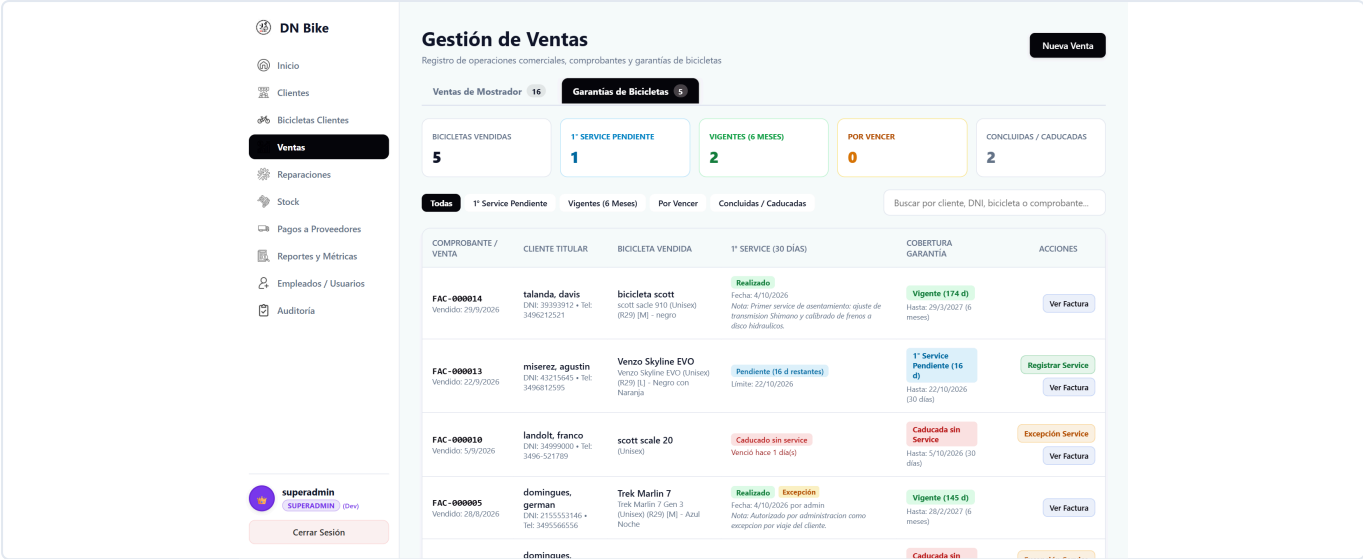


Figura 1.9: Pestaña de Garantías y Gestión de 1º Service Gratuito (90 días)

REGLA DE NEGOCIO

Cada bicicleta nueva vendida genera automáticamente derecho a un 1º service oficial dentro de los 90 días posteriores a la compra.

FUNCIONALIDAD

Monitoreo de estado (Vigente / Vencida / Realizada), registro de realización y opción de excepción autorizada por administrador.

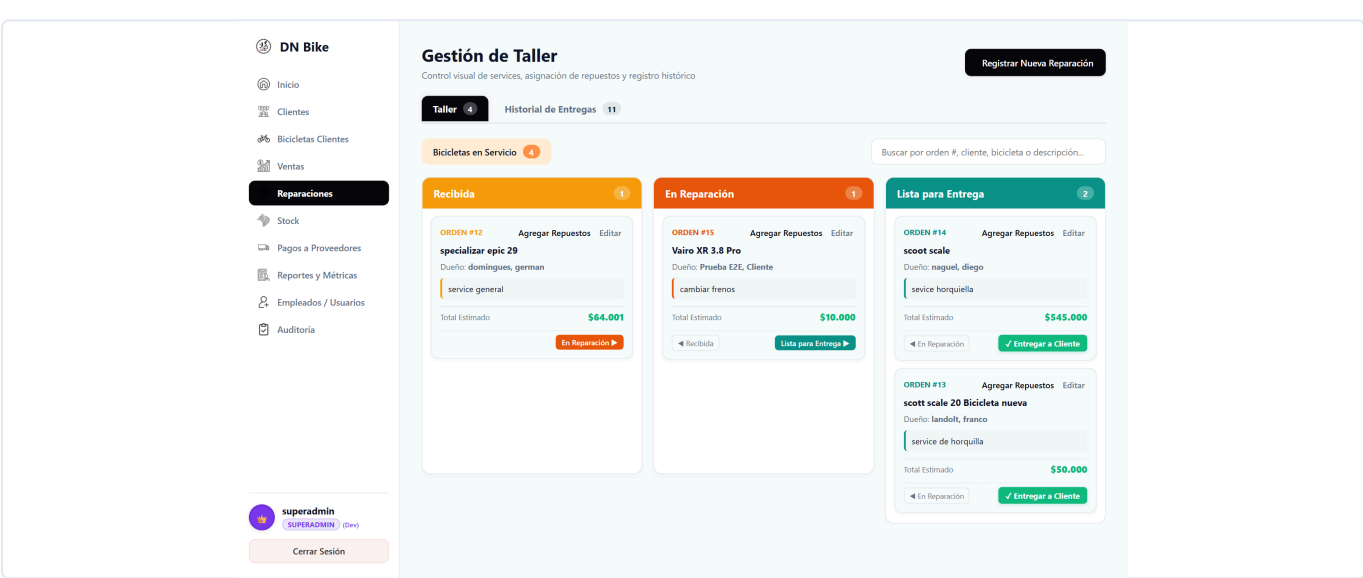


Figura 1.10: Módulo de Reparaciones (Tablero Kanban de Flujo de Trabajo en Taller)

ESTRUCTURA DEL TABLERO

Columnas de estado: Recibida → En Reparación → Lista → Entregada. Permite mover órdenes y visualizar cuellos de botella.

INFORMACIÓN VISIBLE

Tarjeta por rodado con nombre del cliente, descripción de la falla, mecánico asignado, fecha de ingreso y total presupuestado.

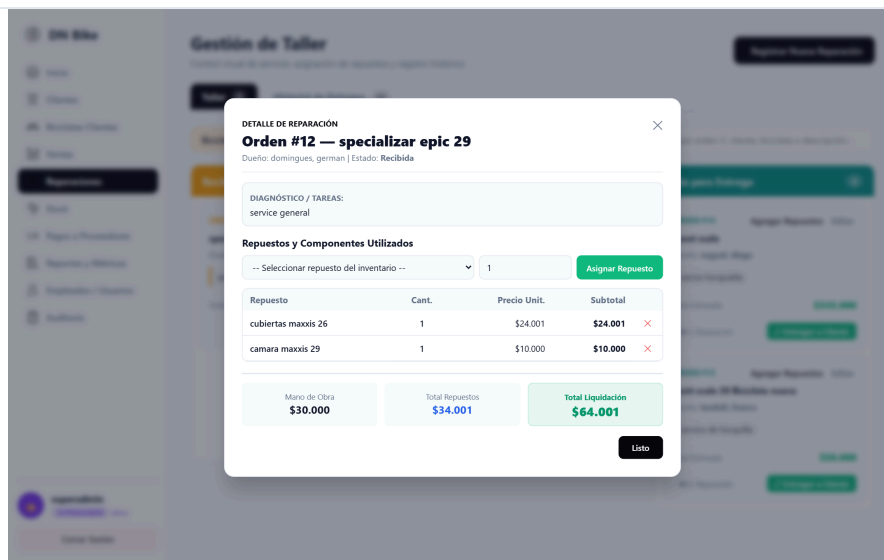


Figura 1.11: Modal de Asignación y Consumo de Repuestos en Reparación

### INTEGRACIÓN CON INVENTARIO

Búsqueda de piezas en stock, validación de cantidad disponible y descuento automático con registro en Movimiento\_Stock.

### CÁLCULO DE COSTO

Suma dinámica del valor de los repuestos utilizados más la mano de obra para calcular el costo total liquidable al cliente.

### DN Bike

Inicio

Cientes

Bicicletas Clientes

Ventas

Reparaciones

**Stock**

Pagos a Proveedores

Reportes y Métricas

Empleados / Usuarios

Auditoría

superadmin (SUPERADMIN) (Dev)

Cerrar Sesión

### Gestión de Inventario y Stock

Control de existencias en tiempo real, catálogo de artículos y movimientos

Movimientos Stock

Ajuste de Stock

Nuevo Producto

ARTÍCULOS EN CATÁLOGO

**79**  
 Variedades de productos registrados

UNIDADES TOTALES

**172**  
 Existencias físicas globales

**BAJO STOCK / ALERTA** **Reabastecer**

**70**  
 Artículos por debajo del stock mínimo

INACTIVOS (BAJA)

**5**  
 No visibles para ventas y taller

Buscar por ID, nombre, marca o talla (ej: '8', 'scott', 'venzo 29', 'talle l')...

Mostrando 79 artículos

Todos los Tipos

Bicicletas

Repuestos

Accesorios

STOCK: Todos

Bajo Stock

Sin Stock

Óptimo

ESTADO: Todos

Activos

Inactivos

| ID   | ARTÍCULO / ESPECIFICACIONES                                                                                                                                                                         | CATEGORÍA | PRECIO VENTA | STOCK / MÍNIMO     | DISPONIBILIDAD | ACCIONES    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|--------------|--------------------|----------------|-------------|
| #151 | forma fxa carbono<br>fsa pro                                                                                                                                                                        | Repuesto  | \$150.000    | 0 unidades / mín 1 | Agotado        | Editar Baja |
| #150 | cuadro y horquilla, palancas durace y manubrio<br>carbono pista look talle M 54<br>Peretti cuadro y horquilla, palancas durace y manubrio car<br>Unisex   Talle M   Rodado 29   Color Negro/Surtido | Bicicleta | \$5.300.120  | 1 unidad / mín 1   | Reabastecer    | Editar Baja |
| #149 | raza L microshift<br>Unisex   Talle M   Rodado 29   Color Negro/Surtido                                                                                                                             | Bicicleta | \$462.875    | 1 unidad / mín 1   | Reabastecer    | Editar Baja |
| #148 | cube c62 elite 1<br>Cube cube c62 elite 1<br>Unisex   Talle M   Rodado 29   Color Negro/Surtido                                                                                                     | Bicicleta | \$3.079.930  | 1 unidad / mín 1   | Reabastecer    | Editar Baja |
| #147 | ruedas crossmark (500)<br>Peretti ruedas crossmark (500)<br>Unisex   Talle M   Rodado 29   Color Negro/Surtido                                                                                      | Bicicleta | \$987.600    | 1 unidad / mín 1   | Reabastecer    | Editar Baja |

Figura 1.12: Módulo de Control de Stock y Catálogo de Productos

### PROPÓSITO OPERATIVO

Gestionar el catálogo unificado de repuestos, accesorios y bicicletas nuevas para venta con sus precios y stock disponible.

### ALERTAS VISUALES

Destacado en color ámbar/rojo de productos que alcanzaron o cayeron por debajo de su stock\_mínimo configurado.

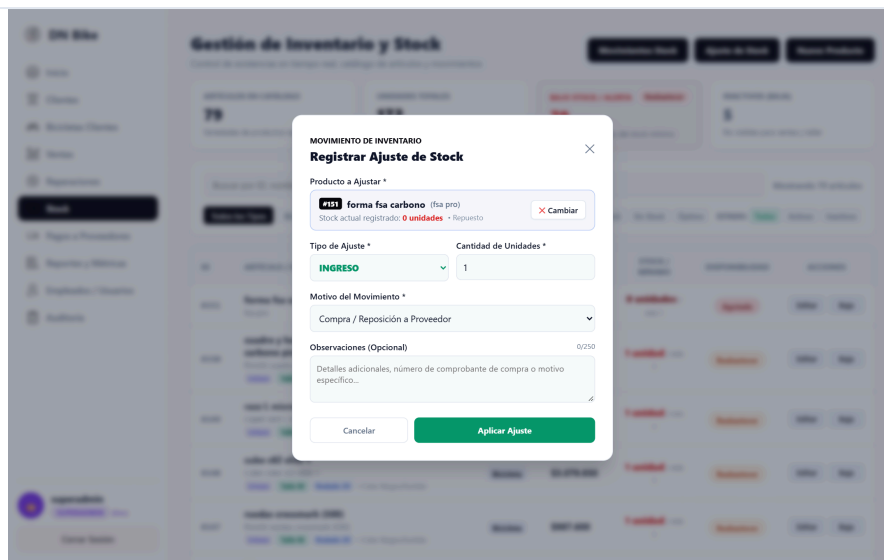


Figura 1.13: Modal Emergente para Ajuste Manual de Stock (Ingreso / Egreso)

#### AUDITORÍA OBLIGATORIA

Exige seleccionar motivo (Inventario físico, rotura, merma, devolución) y descripción obligatoria para auditoría.

#### IMPACTO EN BD

Actualiza Productos . cantidad y genera un registro inmutable en la tabla Movimiento\_Stock.

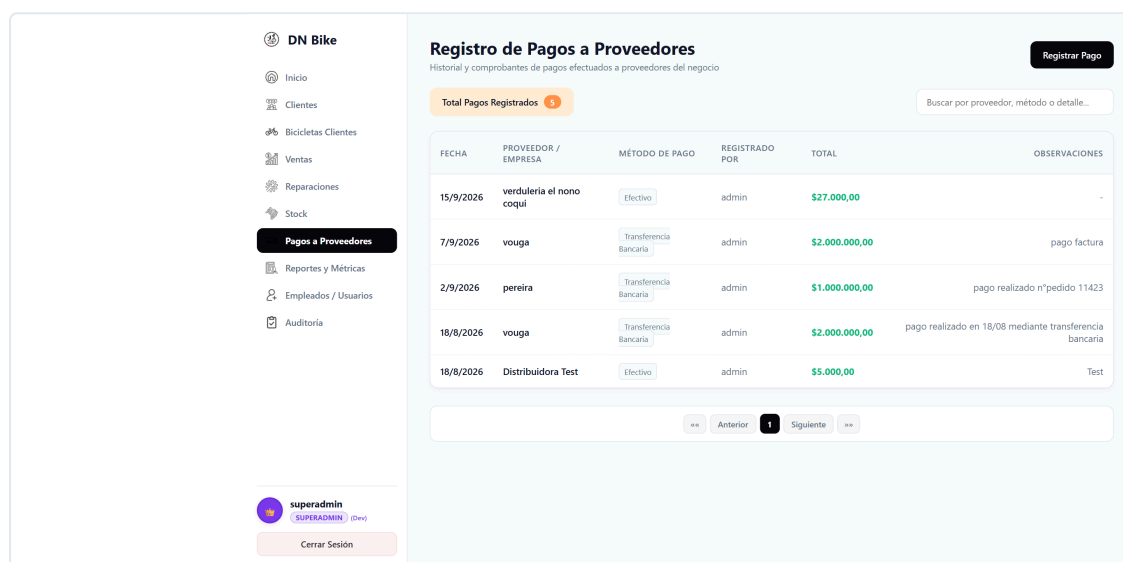


Figura 1.14: Módulo de Proveedores y Registro de Pagos Comerciales

#### PROPÓSITO OPERATIVO

Controlar los distribuidores mayoristas de repuestos y registrar los egresos de caja por compra de mercadería y servicios externos.

#### DATOS REGISTRADOS

Proveedor, fecha de pago, importe total, método de pago (transferencia, cheque, efectivo) y comprobante o factura asociada.

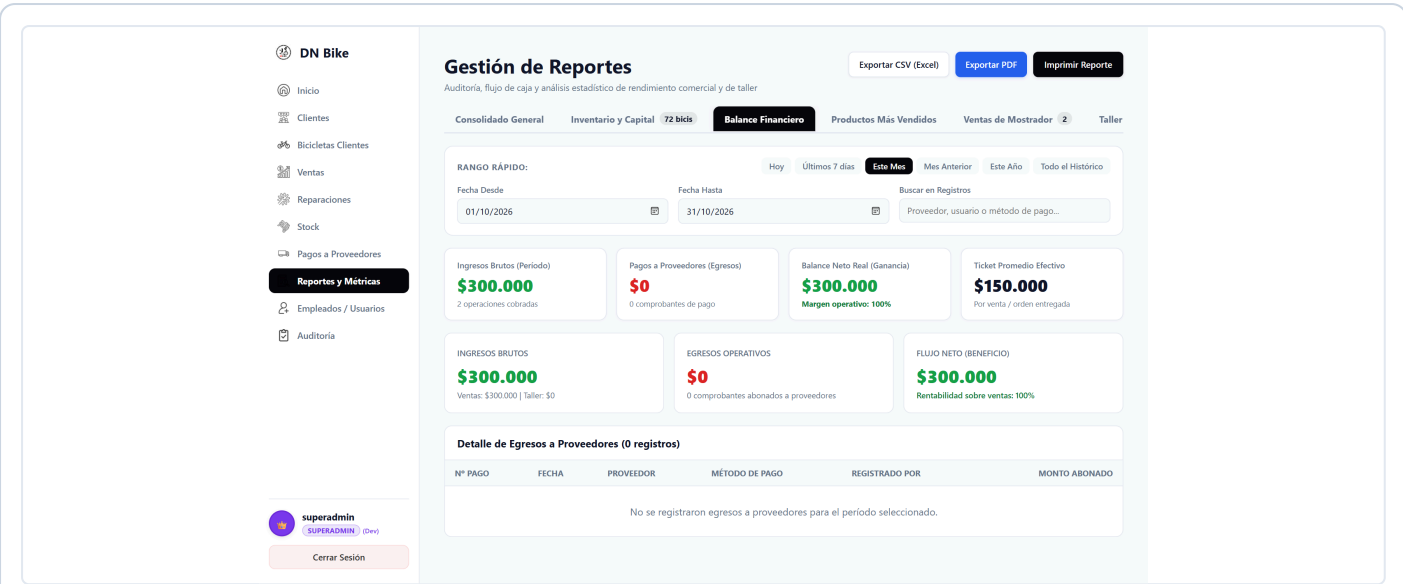


Figura 1.15: Módulo de Reportes Financieros, Ventas y Rentabilidad

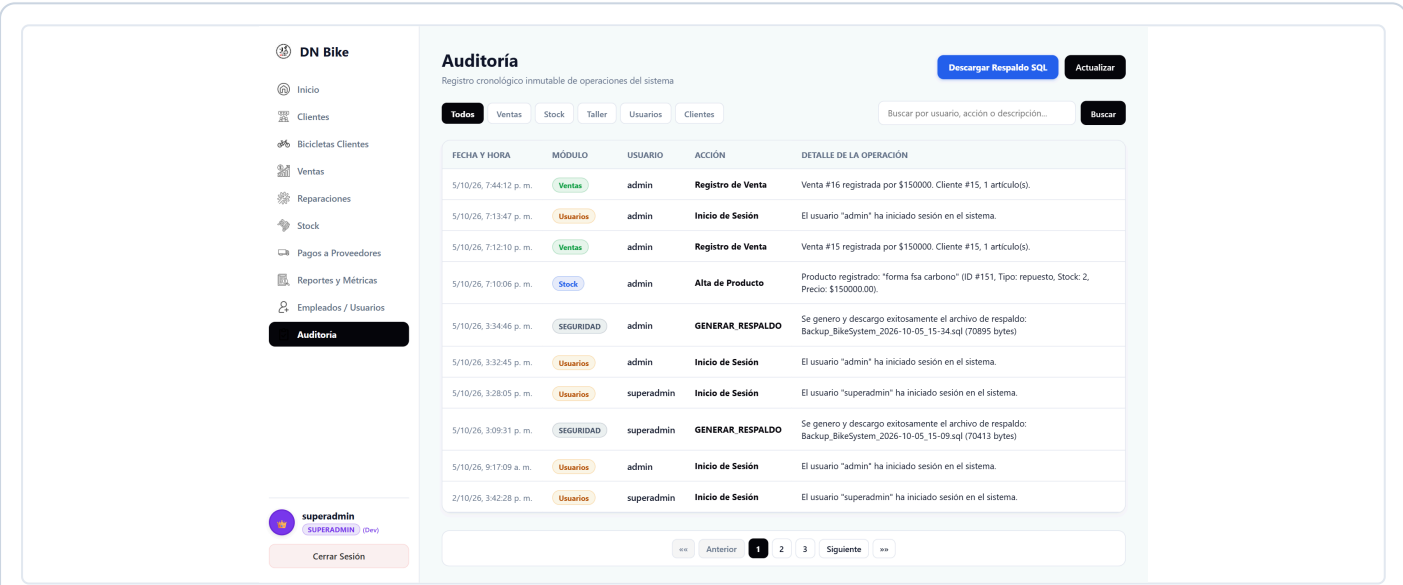
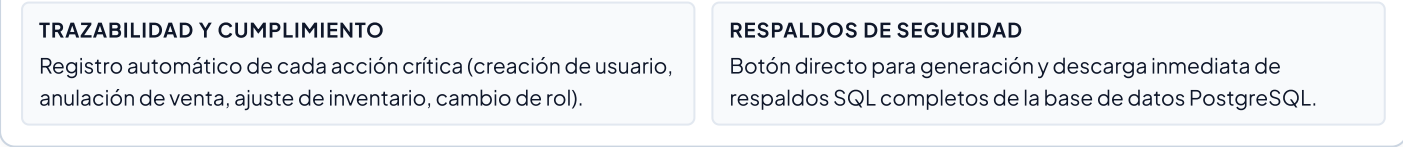


Figura 1.16: Módulo de Seguridad y Bitácora de Auditoría de Actividades



1.4 Diagrama Explícito de Flujo de Navegación entre Pantallas (User Flow / Screen Map)

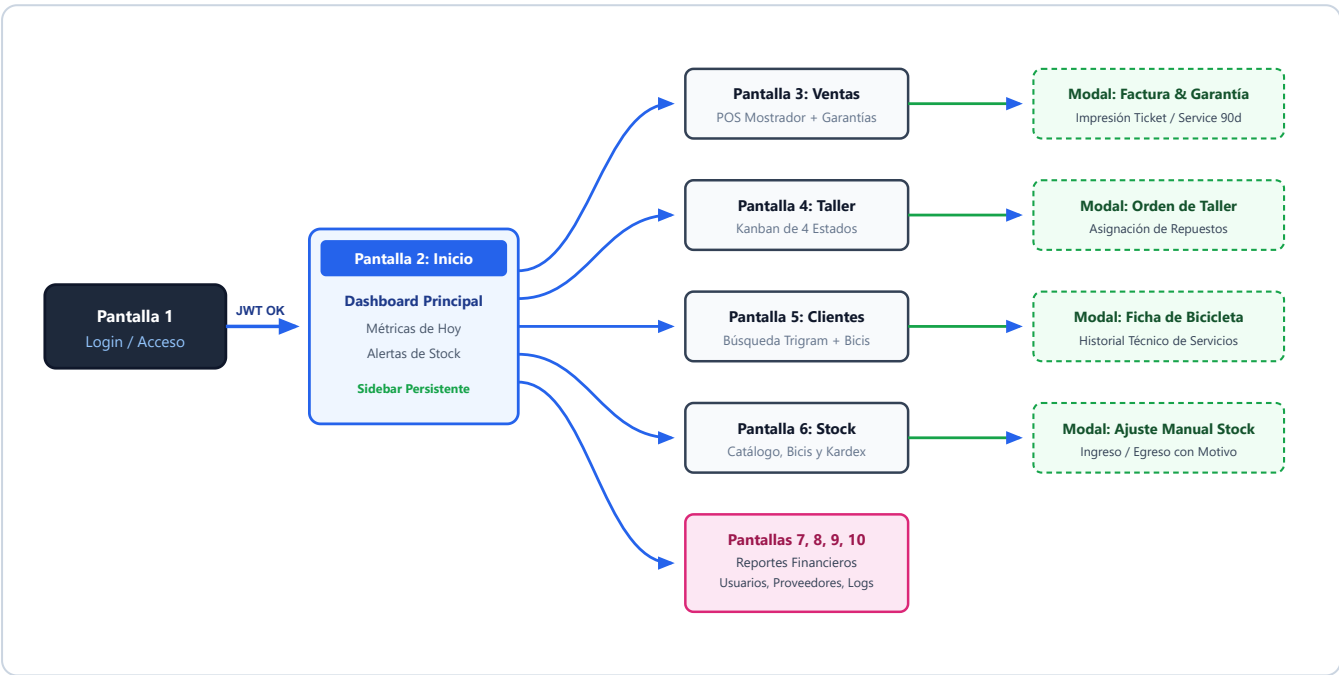
Respuesta Formal a la Observación 1 de la Rúbrica

A continuación se presenta el mapa arquitectónico de pantallas que ilustra la navegación operativa del usuario, los puntos de bifurcación, la persistencia del menú lateral (Sidebar) y las invocaciones a ventanas modales de edición y confirmación.

ARQUITECTURA UX / FLUJO DE NAVEGACIÓN

Diagrama de Flujo de Navegación entre Pantallas

Respuesta explícita a la **Observación 1 de la Rúbrica de Primera Entrega**. Detalla el recorrido del usuario desde la autenticación, la barra de navegación persistente (Sidebar) y las transiciones a modales operativos.



## 1.5 Modelo de Datos: Modelo Lógico y Unificación de Automatización de Stock

### Respuesta Formal a la Observación 2 de la Rúbrica (Criterio Uniforme de Movimientos de Stock)

El docente señaló en la rúbrica: "Veo que el modelo incorpora un trigger para actualizar automáticamente el stock luego de registrar una venta. Sugeriría revisar si resulta conveniente implementar una lógica similar para otros movimientos de stock (por ejemplo, ingresos de mercadería o consumo de repuestos en reparaciones), de manera de mantener un criterio uniforme en todas las operaciones del sistema."

**Solución Técnica Implementada y Justificada:** Para asegurar un criterio absolutamente uniforme y trazable, el equipo implementó una arquitectura basada en la entidad transaccional `Movimiento_Stock` combinada con operaciones atómicas en backend y disparadores de consistencia en PostgreSQL:

- 1. Ventas de Mostrador (Egreso Automático):** Al crearse una venta en `Detalle_Venta`, se descuenta la cantidad en `Productos` y se genera automáticamente un registro en `Movimiento_Stock` con tipo 'EGRESO' y motivo 'Venta #ID'.
- 2. Anulación de Ventas (Ingreso Automático):** Al anular una factura comercial, el sistema reintegra automáticamente las cantidades a `Productos` y asienta un registro de tipo 'INGRESO' con motivo 'Anulación de Venta #ID' y el motivo auditado.
- 3. Taller y Reparaciones (Egreso Automático):** Al agregar repuestos a una orden técnica en `Detalle_Reparacion`, se descuenta de inmediato el stock físico disponible para evitar ventas duplicadas, registrando un `Movimiento_Stock` de tipo 'EGRESO' con motivo 'Consumo en Reparación #ID'. Si la orden se cancela o se retira el repuesto, se restituye mediante 'INGRESO'.
- 4. Ingresos por Compras a Proveedores (Ingreso Automático):** El ingreso de mercadería actualiza las existencias registrando 'INGRESO' vinculado al usuario operador y al proveedor.
- 5. Ajustes Manuales de Auditoría:** Cualquier discrepancia física se regulariza mediante el módulo de stock, registrando obligatoriamente 'INGRESO' o 'EGRESO' con justificación.

## 1.6 Sentencias DDL Oficiales de BikeSystem (PostgreSQL)

A continuación se transcriben las sentencias DDL completas ejecutadas en el motor PostgreSQL 15+, incluyendo extensiones, tipos generados, restricciones de integridad, claves foráneas e índices de alto rendimiento:

```
-- Habilitar extensión oficial de PostgreSQL para búsqueda por similitud y subcadenas masivas
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 1. TABLAS INDEPENDIENTES

CREATE TABLE Usuario (
    id_usuario INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_usuario VARCHAR(50) NOT NULL UNIQUE CONSTRAINT chk_usuario_nombre_min CHECK
(length(trim(nombre_usuario)) ≥ 3),
    contrasena VARCHAR(255) NOT NULL,
    rol VARCHAR(20) NOT NULL CONSTRAINT chk_usuario_rol CHECK (rol IN ('EMPLEADO', 'ADMIN', 'SUPERADMIN')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Cliente (
    id_cliente INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL CONSTRAINT chk_cliente_nombre_min CHECK (length(trim(nombre)) ≥ 2),
    apellido VARCHAR(100) NOT NULL CONSTRAINT chk_cliente_apellido_min CHECK (length(trim(apellido)) ≥ 2),
    dni VARCHAR(20) UNIQUE CONSTRAINT chk_cliente_dni_min CHECK (dni IS NULL OR length(trim(dni)) ≥ 6),
    telefono VARCHAR(20),
    email VARCHAR(100),
    direccion VARCHAR(200),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Proveedor (
    id_proveedor INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_empresa VARCHAR(100) NOT NULL CONSTRAINT chk_prov_nombre_min CHECK (length(trim(nombre_empresa)) ≥
2),
    cuil VARCHAR(20) UNIQUE,
```

```

telefono VARCHAR(20),
email VARCHAR(100),
direccion VARCHAR(200),
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Productos (
    id_producto INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL CONSTRAINT chk_prod_nombre_min CHECK (length(trim(nombre)) ≥ 2),
    marca VARCHAR(50),
    modelo VARCHAR(50) NULL,
    tipo_prod VARCHAR(50) NOT NULL CONSTRAINT chk_prod_tipo_valido CHECK (LOWER(tipo_prod) IN ('bicicleta',
'repuesto', 'accesorio')),
    cantidad INT DEFAULT 0 CONSTRAINT chk_productos_cantidad CHECK (cantidad ≥ 0),
    precio DECIMAL(10, 2) DEFAULT 0 CONSTRAINT chk_productos_precio CHECK (precio ≥ 0),
    stock_minimo INT DEFAULT 0 CONSTRAINT chk_productos_stock_minimo CHECK (stock_minimo ≥ 0),
    activo BOOLEAN NOT NULL DEFAULT true,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Metodo_Pago (
    id_metodo_pago INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

-- 2. TABLAS DEPENDIENTES Y TRANSACCIONALES

CREATE TABLE Bicicleta (
    id_bicicleta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_cliente INT NOT NULL,
    marca VARCHAR(50) NOT NULL CONSTRAINT chk_bici_marca_min CHECK (length(trim(marca)) ≥ 1),
    modelo VARCHAR(50),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_bici_cliente FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente) ON DELETE RESTRICT
);

CREATE TABLE Venta (
    id_venta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_usuario INT NOT NULL,
    id_metodo_pago INT NOT NULL DEFAULT 1,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    costo_total DECIMAL(10, 2) NOT NULL DEFAULT 0 CONSTRAINT chk_venta_costo_total CHECK (costo_total ≥ 0),
    estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA' CONSTRAINT chk_venta_estado CHECK (estado IN ('COMPLETADA',
'ANULADA')),
    fecha_anulacion TIMESTAMPTZ NULL,
    motivo_anulacion TEXT NULL,
    cliente_nombre VARCHAR(100) NULL,
    cliente_apellido VARCHAR(100) NULL,
    cliente_dni VARCHAR(20) NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_venta_cliente FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente) ON DELETE RESTRICT,
    CONSTRAINT fk_venta_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT,
    CONSTRAINT fk_venta_metodo FOREIGN KEY (id_metodo_pago) REFERENCES Metodo_Pago(id_metodo_pago) ON DELETE
RESTRICT
);

CREATE TABLE Reparacion (
    id_reparacion INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_bicicleta INT NOT NULL,
    id_usuario INT NOT NULL,
    fecha_ingreso DATE NOT NULL DEFAULT CURRENT_DATE,
    fecha_egreso DATE,
    estado VARCHAR(50) NOT NULL CONSTRAINT chk_reparacion_estado_valido CHECK (estado IN ('Recibida', 'En
Reparación', 'Lista', 'Entregada')),
    descripcion TEXT,

```

```

costo_manoobra DECIMAL(10, 2) DEFAULT 0 CONSTRAINT chk_reparacion_manoobra CHECK (costo_manoobra ≥ 0),
costo_total DECIMAL(10, 2) DEFAULT 0 CONSTRAINT chk_reparacion_costo_total CHECK (costo_total ≥ 0),
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_rep_bicicleta FOREIGN KEY (id_bicicleta) REFERENCES Bicicleta(id_bicicleta) ON DELETE
RESTRICT,
CONSTRAINT fk_rep_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT
);

CREATE TABLE Detalle_Venta (
    id_detalle_venta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_venta INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL CONSTRAINT chk_detventa_cantidad CHECK (cantidad > 0),
    precio_unitario DECIMAL(10, 2) NOT NULL,
    costo_total DECIMAL(10, 2) NOT NULL,
    primer_service_realizado BOOLEAN NOT NULL DEFAULT false,
    fecha_primer_service DATE NULL,
    observaciones_service TEXT NULL,
    id_usuario_service INT NULL,
    service_excepcion BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT fk_detventa_venta FOREIGN KEY (id_venta) REFERENCES Venta(id_venta) ON DELETE CASCADE,
    CONSTRAINT fk_detventa_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE RESTRICT,
    CONSTRAINT fk_detventa_usuario_service FOREIGN KEY (id_usuario_service) REFERENCES Usuario(id_usuario) ON
DELETE SET NULL
);

CREATE TABLE Detalle_Reparacion (
    id_detalle_rep INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_reparacion INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL CONSTRAINT chk_detrep_cantidad CHECK (cantidad > 0),
    precio_unitario DECIMAL(10, 2) NOT NULL,
    costo_total DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_detrep_rep FOREIGN KEY (id_reparacion) REFERENCES Reparacion(id_reparacion) ON DELETE CASCADE,
    CONSTRAINT fk_detrep_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE RESTRICT
);

CREATE TABLE Movimiento_Stock (
    id_movimiento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_producto INT NOT NULL,
    id_usuario INT NOT NULL,
    tipo_movimiento VARCHAR(20) NOT NULL CONSTRAINT chk_mov_tipo CHECK (tipo_movimiento IN ('INGRESO',
'EGRESO')),
    cantidad INT NOT NULL CONSTRAINT chk_mov_cant CHECK (cantidad > 0),
    motivo VARCHAR(100) NOT NULL,
    observaciones TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_mov_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE RESTRICT,
    CONSTRAINT fk_mov_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT
);

```

## 2 Arquitectura del Sistema

### 2.1 Visión General del Stack Tecnológico

BikeSystem fue diseñado e implementado siguiendo un esquema moderno de aplicación de escritorio empresarial de alto rendimiento, combinando la velocidad de desarrollo del ecosistema web con la seguridad y persistencia nativa del sistema operativo:

| Capa Tecnológica | Tecnología / Librería       | Versión            | Rol y Justificación Arquitectónica                                                                          |
|------------------|-----------------------------|--------------------|-------------------------------------------------------------------------------------------------------------|
| Desktop Shell    | Electron                    | v42+               | Empaquetador nativo multiplataforma en Windows con control de ventanas y ciclo de vida de escritorio.       |
| Frontend UI      | React + TypeScript + Vite   | React 19, TS 5     | Renderizado reactivo de interfaces, tipado estático riguroso y empaquetado ultraveloz mediante Vite HMR.    |
| Estilos y Diseño | Tailwind CSS + Lucide Icons | Tailwind v3+       | Sistema de diseño profesional con variables CSS personalizadas e iconografía SVG institucional.             |
| Backend API      | Express + Node.js           | Express 5, Node 24 | Servidor HTTP REST modular, desacoplado y preparado para funcionar localmente o en red de área local (LAN). |
| Base de Datos    | PostgreSQL + pg Driver      | PG 15+, pg Pool    | Motor relacional ACID con pool de conexiones concurrentes y extensión pg_trgm para búsquedas de texto.      |
| Seguridad        | JWT + Bcrypt.js             | JWT 9+, Bcrypt 2+  | Tokens criptográficos firmados con HMAC-SHA256 y hashing de claves unidireccional con 10 rondas de salt.    |

### 2.2 Arquitectura en Capas (Clean / Layered Architecture)

La solución implementa una estricta separación de responsabilidades dividida en 4 capas desacopladas:

- **1. Capa de Presentación (Frontend):** Aplicación React 19 ejecutada dentro del proceso Renderer de Electron. Gestiona estados locales, formularios con validación inmediata, componentes reutilizables y comunicación asíncrona mediante un servicio HTTP centralizado ('api.ts').
- **2. Capa de Enrutamiento y Controladores (Backend Controllers):** Recibe las peticiones HTTP, extrae parámetros de ruta y cuerpo, ejecuta validaciones de tipos de datos ('validarId', validación de esquemas) y delega la ejecución a los servicios correspondientes.
- **3. Capa de Servicios y Reglas de Negocio (Backend Services):** Contiene la lógica nuclear de la empresa: verificación de disponibilidad de stock, cálculo de comisiones y totales, coordinación de transacciones atómicas con `BEGIN ... COMMIT ... ROLLBACK` y auditoría de eventos.
- **4. Capa de Acceso a Datos y Persistencia (Database Pool):** Mantiene un grupo administrado de conexiones (Pool) hacia PostgreSQL, ejecuta consultas SQL parametrizadas para evitar inyección SQL y delega en el motor las restricciones de integridad y triggers.

## 2.3 Estructura de Directorios del Monorepo

El proyecto está estructurado como un monorepo administrado con `pnpm workspaces`, asegurando total coherencia en dependencias y compilación de paquetes compartidos:

BikeSystem Monorepo • Arquitectura de Directorios (pnpm workspaces)

produccion-v2.0



3 Diagramas UML Formales

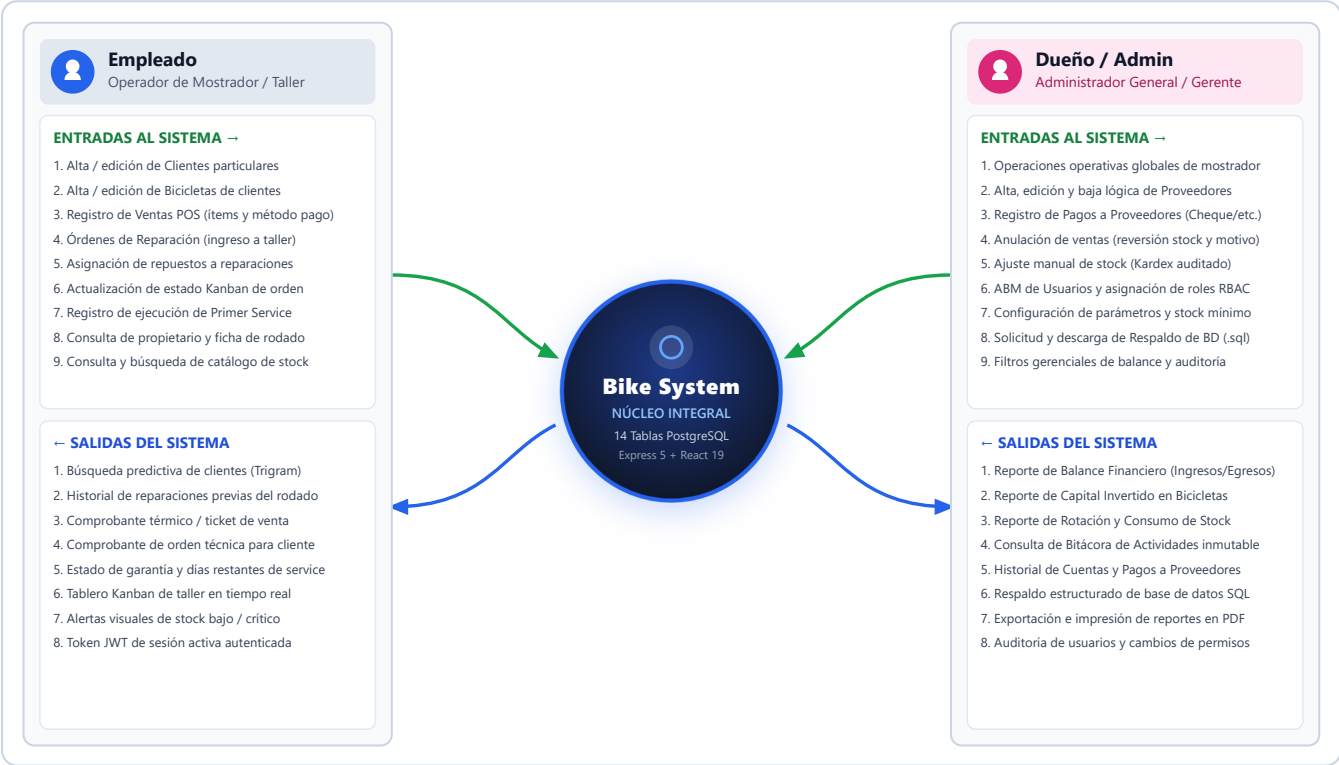
Esta sección presenta los diagramas formales requeridos por la cátedra de Ingeniería de Software (Contexto, Casos de Uso, Diagrama de Clases y Entidad-Relación Peter Chen, además del flujo navegacional de pantallas). Siguiendo fielmente las pautas y modelos de BikeSystem.pdf (páginas 8, 9, 10, 33 y 34), han sido construidos mediante especificación vectorial SVG pura de alta resolución para garantizar total nitidez, tipografía visible, multiplicidades cardinales y relaciones explícitas.

3.1 Diagrama de Contexto del Sistema (Ref: BikeSystem.pdf pág. 8)

UML / DFD NIVEL 0 VECTORIAL

Diagrama de Contexto del Sistema BikeSystem (Integral)

Representa los límites del software, los actores externos (Empleado y Dueño/Admin) y todos los flujos de datos bidireccionales con el núcleo central PostgreSQL + Express (Ref: BikeSystem.pdf pág. 8).

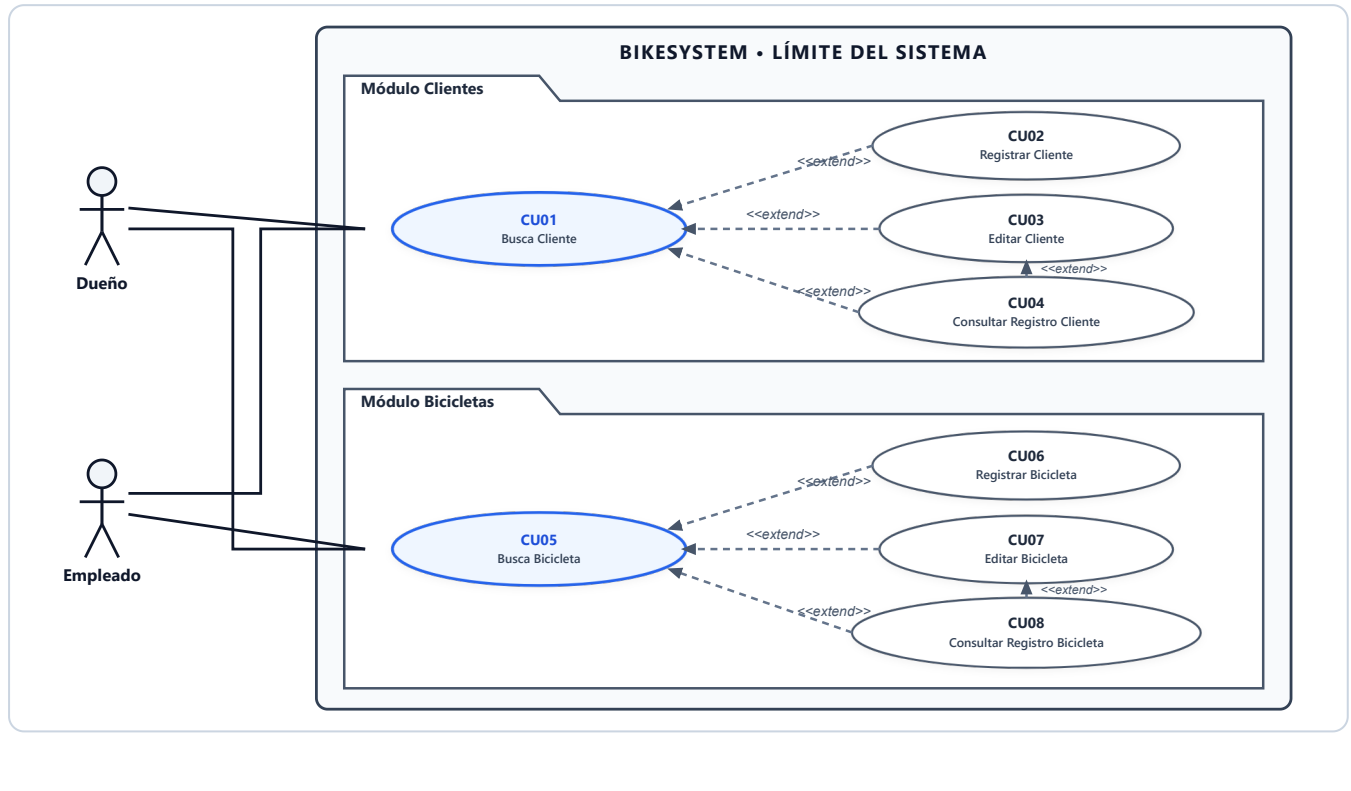


### 3.2.1 Diagrama de Casos de Uso: Módulos Clientes y Bicicletas (Ref: BikeSystem.pdf pág. 8)

UML 2.5 VECTORIAL • LÁMINA 1 DE 4

#### Diagrama de Casos de Uso: Módulos Clientes y Bicicletas

Especifica los casos de uso CU01 a CU08, relaciones <<extend>> y actores Empleado y Dueño (Ref: BikeSystem.pdf pág. 8).

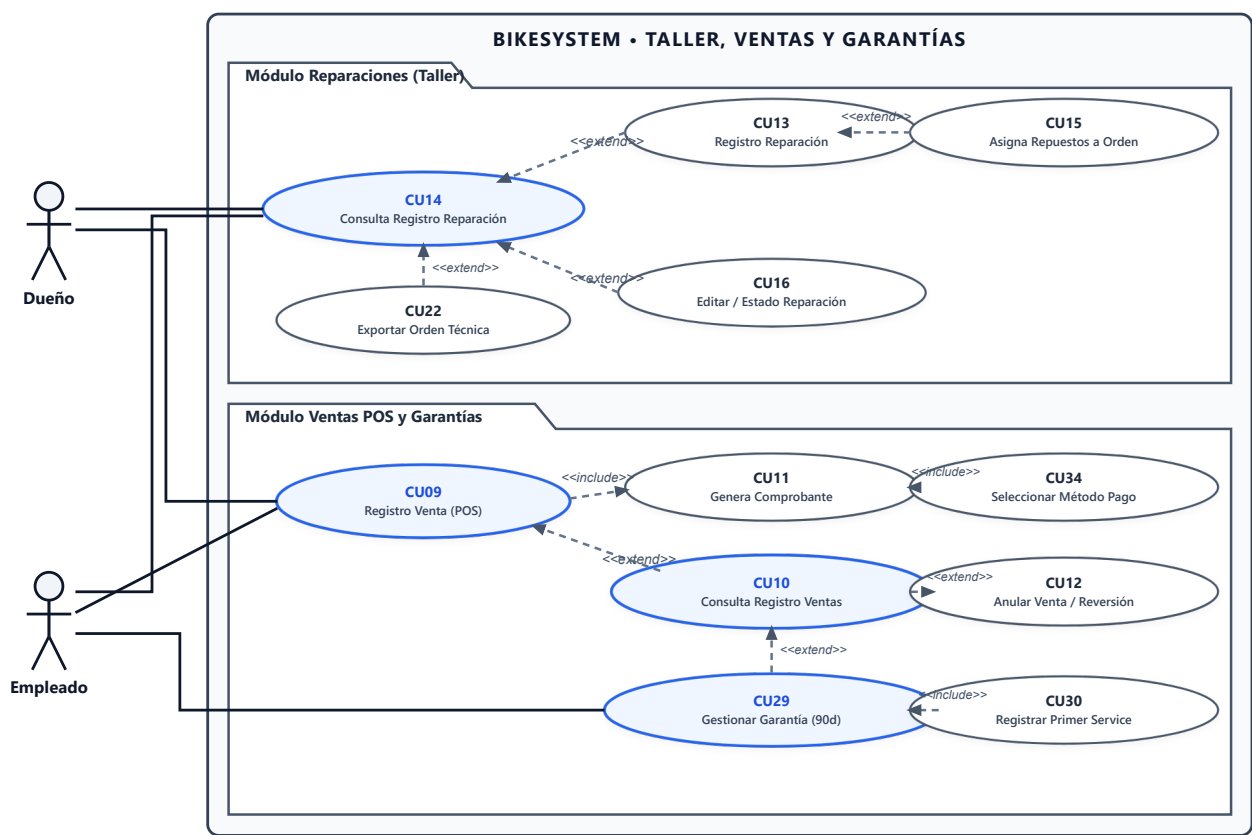


3.2.2 Diagrama de Casos de Uso: Módulos Reparaciones (Taller), Ventas y Garantías (Ref: BikeSystem.pdf pág. 9)

UML 2.5 VECTORIAL • LÁMINA 2 DE 4

Diagrama de Casos de Uso: Módulos Reparaciones (Taller), Ventas y Garantías

Casos de uso del ciclo comercial y técnico: órdenes de trabajo, repuestos, facturación POS, métodos de pago, anulación de ventas y garantías post-venta (Ref: BikeSystem.pdf pág. 9).

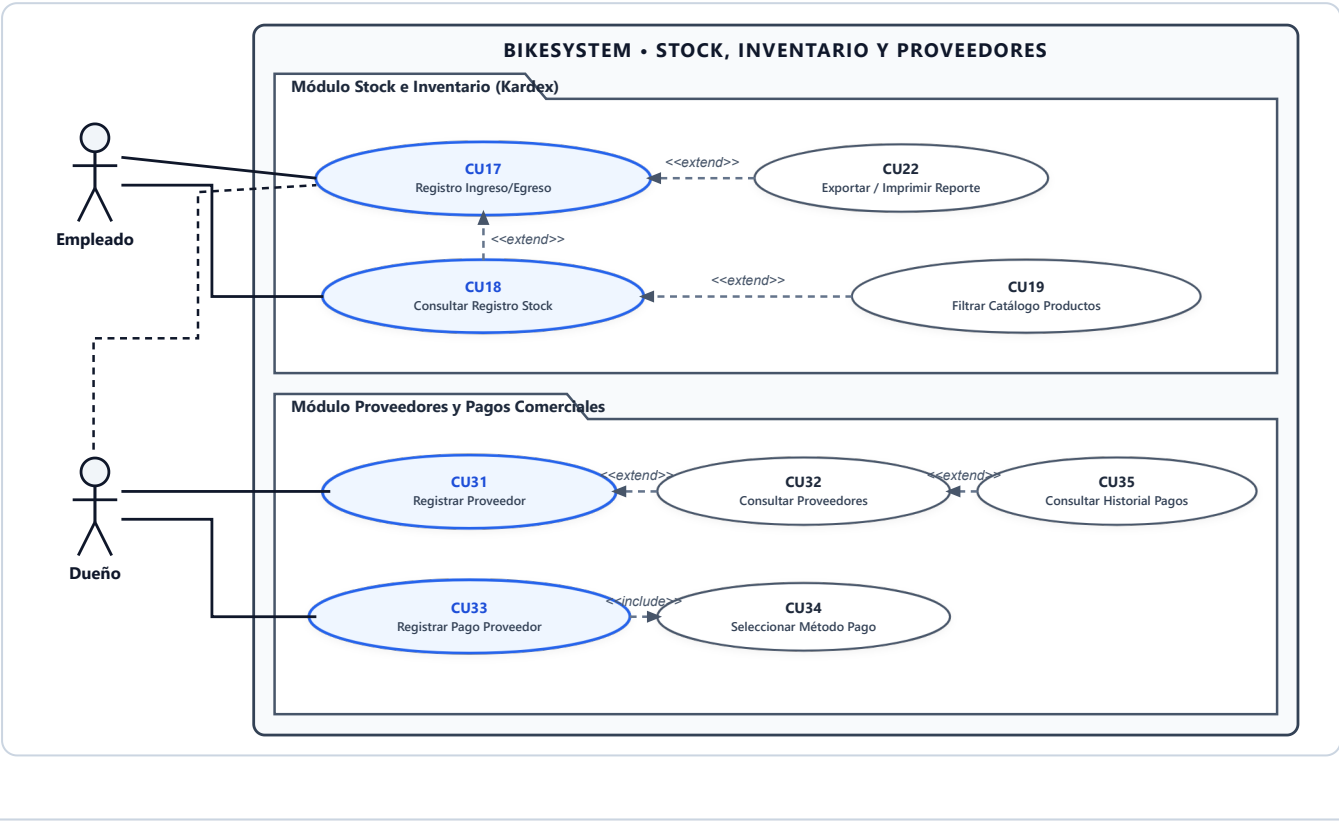


3.2.3 Diagrama de Casos de Uso: Módulos Stock / Kardex y Proveedores / Pagos (Ref: BikeSystem.pdf pág. 9)

UML 2.5 VECTORIAL • LÁMINA 3 DE 4

Diagrama de Casos de Uso: Módulos Stock / Kardex y Proveedores / Pagos

Cubre el control físico de inventario mediante Movimiento\_Stock, alertas de stock mínimo, gestión de proveedores y egresos por pagos (Ref: BikeSystem.pdf pág. 9).

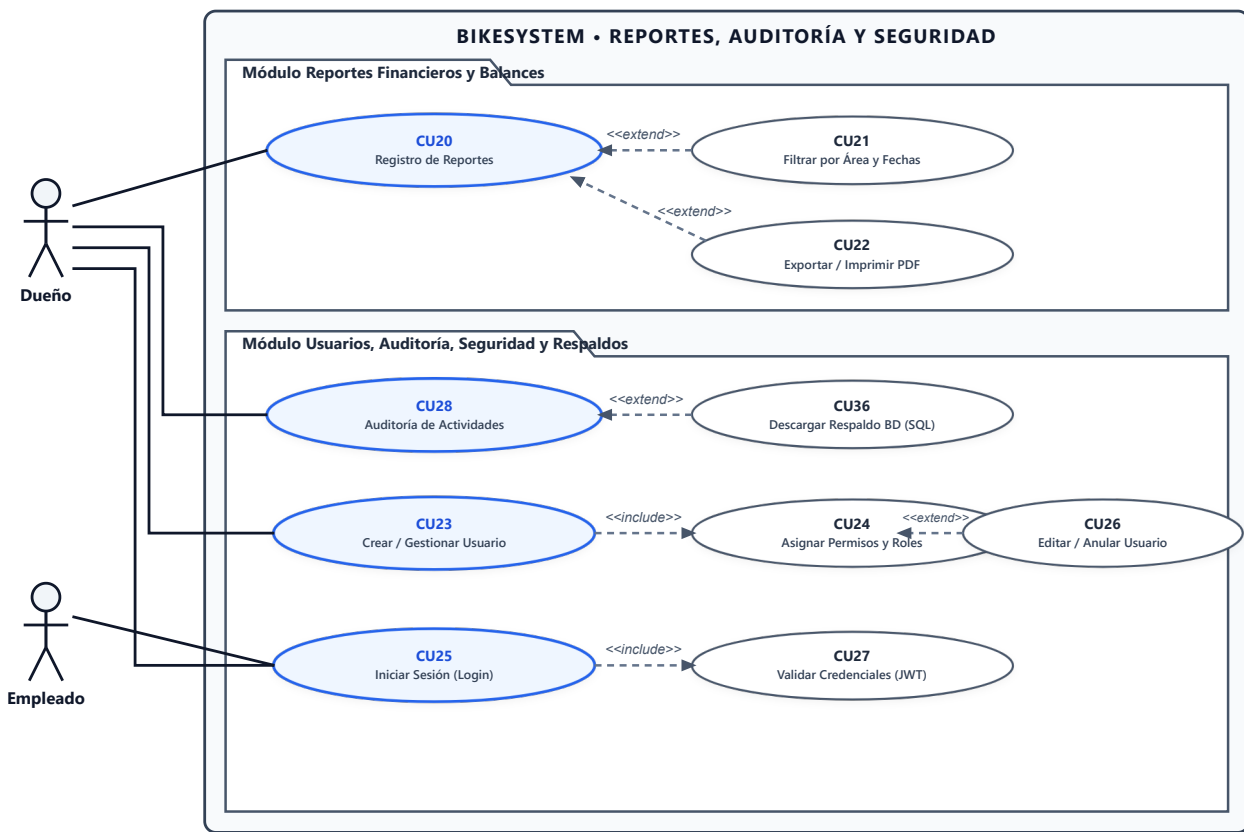


3.2.4 Diagrama de Casos de Uso: Módulos Reportes, Auditoría, Seguridad y Respaldos (Ref: BikeSystem.pdf pág. 10)

UML 2.5 VECTORIAL • LÁMINA 4 DE 4

Diagrama de Casos de Uso: Módulos Reportes, Auditoría, Seguridad y Respaldos

Cubre reportes gerenciales de balance, auditoría inmutable de bitácora, login con tokens JWT, administración de usuarios y respaldos de base de datos (Ref: BikeSystem.pdf pág. 10).

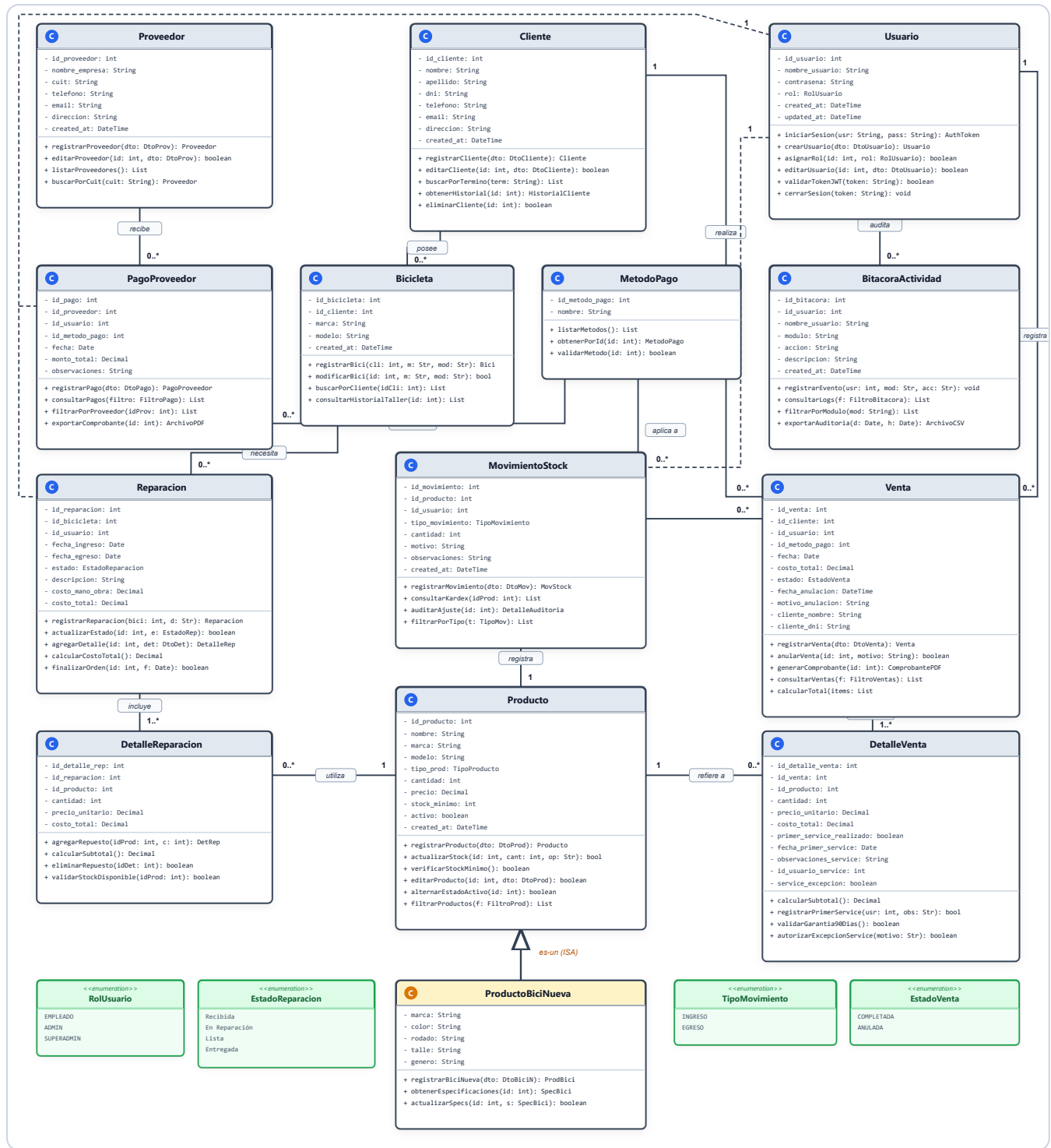


### 3.3 Diagrama de Clases UML del Sistema (14 Clases de Dominio) (Ref: BikeSystem.pdf pág. 34)

#### UML 2.5 VECTORIAL • ARQUITECTURA COMPLETA

## Diagrama de Clases UML del Sistema (14 Clases de Dominio + 4 Enumeraciones)

Estructura formal con 3 compartimentos (Nombre, Atributos tipados y Métodos con firmas completas de parámetros y retornos), composiciones fuertes (◆), generalización de bicicletas nuevas, enumeraciones de dominio y multiplicidades rigurosas (Ref: BikeSystem.pdf pág. 34).



### 3.3.1 Matriz de Relaciones y Multiplicidades del Diagrama de Clases (14 Clases)

| Matriz de Relaciones y Multiplicidades del Diagrama de Clases (14 Clases) |               |                             |               |                   |                                                                                      |
|---------------------------------------------------------------------------|---------------|-----------------------------|---------------|-------------------|--------------------------------------------------------------------------------------|
| Clase Origen                                                              | Multiplicidad | Tipo / Rol                  | Multiplicidad | Clase Destino     | Descripción y Regla de Negocio                                                       |
| Cliente                                                                   | 1             | posee<br>(Asociación)       | 1..*          | Bicicleta         | Un cliente particular posee uno o varios rodados registrados para reparación.        |
| Cliente                                                                   | 1             | realiza<br>(Asociación)     | 0..*          | Venta             | Un cliente efectúa compras de mostrador (soporta cliente Consumidor Final).          |
| Bicicleta                                                                 | 1             | necesita<br>(Asociación)    | 0..*          | Reparacion        | Un rodado registrado en el sistema ingresa al taller para servicios técnicos.        |
| Usuario                                                                   | 1             | registra<br>(Asociación)    | 0..*          | Reparacion        | Un empleado/mecánico autenticado registra y actualiza la orden técnica.              |
| Usuario                                                                   | 1             | registra<br>(Asociación)    | 0..*          | Venta             | Un vendedor autenticado emite y factura la operación en el mostrador POS.            |
| Usuario                                                                   | 1             | emite<br>(Asociación)       | 0..*          | PagoProveedor     | El Dueño o administrador registra una orden de pago por compras a proveedores.       |
| Usuario                                                                   | 1             | autoriza<br>(Asociación)    | 0..*          | MovimientoStock   | Cada ingreso, egreso o ajuste físico queda auditado con el ID del operador.          |
| Usuario                                                                   | 1             | audita<br>(Asociación)      | 0..*          | BitacoraActividad | Cada acción crítica efectuada genera un registro inmutable en bitácora.              |
| Reparacion                                                                | 1             | incluye<br>(Composición ♦)  | 0..*          | DetalleReparacion | La orden de trabajo compone sus repuestos; la eliminación en cascada revierte stock. |
| Venta                                                                     | 1             | contiene<br>(Composición ♦) | 1..*          | DetalleVenta      | La venta compone sus ítems comercializados; la anulación restituye existencias.      |
| DetalleReparacion                                                         | 0..*          | utiliza<br>(Asociación)     | 1             | Producto          | Cada renglón de repuesto aplicado                                                    |

| Clase Origen        | Multiplicidad | Tipo / Rol                | Multiplicidad | Clase Destino            | Descripción y Regla de Negocio                                                                  |
|---------------------|---------------|---------------------------|---------------|--------------------------|-------------------------------------------------------------------------------------------------|
|                     |               |                           |               |                          | descuenta stock físico del inventario.                                                          |
| <b>DetalleVenta</b> | 0..*          | refiere a (Asociación)    | 1             | <b>Producto</b>          | Cada ítem facturado descuenta stock y habilita garantía si es bicicleta nueva.                  |
| <b>MetodoPago</b>   | 1             | aplica a (Asociación)     | 0..*          | <b>Venta</b>             | Modalidad de cobro (Efectivo, Transferencia, Cheque o Mercado Pago).                            |
| <b>Proveedor</b>    | 1             | recibe (Asociación)       | 0..*          | <b>PagoProveedor</b>     | Proveedor mayorista que percibe cancelaciones de compras y repuestos.                           |
| <b>MetodoPago</b>   | 1             | acredita en (Asociación)  | 0..*          | <b>PagoProveedor</b>     | Canal de pago utilizado para liquidar órdenes a proveedores comerciales.                        |
| <b>Producto</b>     | 1             | registra (Asociación)     | 0..*          | <b>MovimientoStock</b>   | Trazabilidad Kardex unificada para compras, ventas, mermas y reparaciones.                      |
| <b>Producto</b>     | 1             | hereda (Generalización ▲) | 0..1          | <b>ProductoBiciNueva</b> | Especialización: las bicicletas nuevas para venta extienden Producto con rodado, talle y color. |

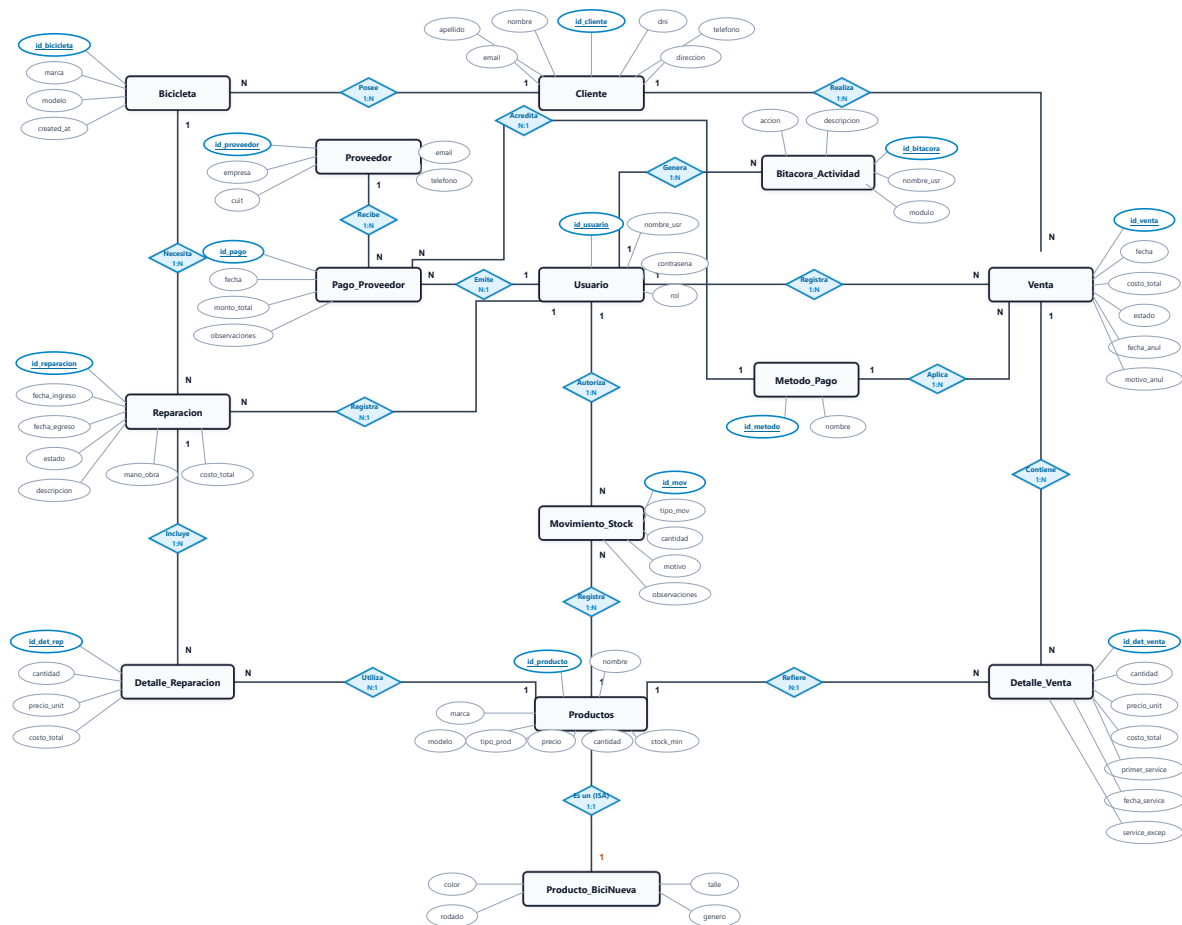
### 3.4 Diagrama Entidad-Relación – Modelo Conceptual Peter Chen (Esquema Unificado del Sistema)

A continuación se presenta el **Diagrama Entidad-Relación formal en notación Peter Chen** que unifica las **14 tablas relacionales** de la base de datos PostgreSQL de BikeSystem en un único lienzo conceptual. El modelo refleja con rigurosidad académica la totalidad de entidades, atributos elípticos, claves primarias subrayadas (PK), rombos asociativos con nombres de rol y cardinalidades normalizadas (1:1, 1:N, N:1), garantizando trazabilidad completa con el esquema DDL oficial (database/bikesystem.sql) y los lineamientos evaluativos de cátedra (Ref: *BikeSystem.pdf* pág. 33):

#### MODELO CONCEPTUAL PETER CHEN • ESQUEMA UNIFICADO

### Diagrama Entidad-Relación Peter Chen Unificado (14 Tablas de PostgreSQL)

Esquema maestro unificado de la base de datos: 14 entidades rectangulares, rombos con cardinalidades 1:N / 1:1, elipses con atributos completos, claves primarias (PK) subrayadas y trazado ortogonal sin cruces (Ref: *BikeSystem.pdf* pág. 33).



## 4 Descripción Técnica de Funcionalidades Implementadas

A continuación se detalla exhaustivamente la implementación técnica de cada uno de los 10 módulos del sistema, describiendo su comportamiento operativo, endpoints REST asociados, reglas de negocio y mecanismos de persistencia:

### 4.1 Módulo 1: Autenticación, Seguridad y Roles (RBAC)

**Qué hace:** Controla el ingreso de usuarios mediante credenciales seguras, emite tokens de sesión JWT y autoriza peticiones clasificándolas entre los roles ADMIN (Dueño) y EMPLEADO (Vendedor/Mecánico).

**Cómo lo hace:** Al enviar POST `/api/auth/login`, el backend busca el usuario en la tabla `Usuario`, compara la contraseña en texto plano contra el hash almacenado mediante `bcrypt.compare()`. Si es válida, firma un JWT con payload conteniendo `id`, `nombre_usuario` y `rol`. En cada petición posterior, el middleware `autenticarToken` verifica la firma y añade el usuario al objeto `req`. Rutas administrativas sensibles (usuarios, respaldos) aplican el middleware `requerirRol([ 'ADMIN' ])`.

| Método | Endpoint                       | Rol Requerido | Descripción                            |
|--------|--------------------------------|---------------|----------------------------------------|
| POST   | <code>/api/auth/login</code>   | Público       | Autenticación y generación de JWT      |
| GET    | <code>/api/usuarios</code>     | ADMIN         | Listado de cuentas operadoras          |
| POST   | <code>/api/usuarios</code>     | ADMIN         | Creación de usuario con Bcrypt salt 10 |
| PUT    | <code>/api/usuarios/:id</code> | ADMIN         | Modificación o bloqueo de credenciales |

### 4.2 Módulo 2: Gestión de Clientes y Consumidor Final

**Qué hace:** Administra el padrón de clientes particulares y corporativos, permitiendo búsquedas rápidas en mostrador y dando soporte nativo al cliente anónimo 'Consumidor Final'.

**Cómo lo hace:** El backend expone GET `/api/clientes?busqueda=...`. En la base de datos se utiliza la función `trigram similarity()` de la extensión `pg_trgm` sobre las columnas `nombre`, `apellido` y `dni`, devolviendo coincidencias instantáneas incluso con errores tipográficos. Para ventas rápidas de mostrador sin datos del cliente, el sistema utiliza un registro comodín preexistente de 'Consumidor Final' evitando violaciones de clave foránea.

### 4.3 Módulo 3: Parque Rodado y Bicicletas de Clientes

**Qué hace:** Registra las bicicletas de los clientes asignándoles número de cuadro/serie, marca, modelo y características técnicas para vincularlas a su historial de reparaciones.

**Cómo lo hace:** A través de POST `/api/bicicletas` se valida la existencia del `id_cliente` con restricción ON DELETE RESTRICT. El endpoint GET `/api/bicicletas/:id/historial` realiza un JOIN con `Reparacion` y `Detalle_Reparacion`, permitiendo al mecánico visualizar en un modal cada intervención previa y pieza sustituida.

### 4.4 Módulo 4: Control de Stock, Catálogo y Alertas Críticas

**Qué hace:** Controla las existencias de repuestos, accesorios y bicicletas nuevas para venta. Advierte cuando las unidades caen por debajo del umbral mínimo y audita ajustes manuales.

**Cómo lo hace:** Al invocar POST `/api/productos/ajuste-stock`, el servicio inicia una transacción: actualiza `Productos.cantidad` con validación CHECK (`cantidad >= 0`) e inserta una fila en `Movimiento_Stock` con el motivo auditado y el ID del operador. La consulta de catálogo filtra por defecto productos activos y utiliza un índice B-Tree sobre (`cantidad <= stock_minimo`) para alimentar las alertas del dashboard.

#### 4.5 Módulo 5: Punto de Venta (POS), Facturación y Métodos de Pago

**Qué hace:** Procesa operaciones de venta comercial, valida stock en tiempo real, descuenta cantidades del inventario y genera el comprobante térmico. Permite la anulación justificada de ventas.

**Cómo lo hace:** POST /api/ventas ejecuta una transacción ACID con `pool.connect()`:

1. Inserta la cabecera en Venta calculando subtotales y vinculando el `id_metodo_pago`.
2. Para cada ítem, inserta en `Detalle_Venta` y descuenta stock con `UPDATE Productos SET cantidad = cantidad - $1 WHERE id_producto = $2`.
3. Genera automáticamente el registro `Movimiento_Stock` (tipo 'EGRESO'). Si alguna línea no dispone de existencias suficientes, lanza una excepción y revierte toda la operación (ROLLBACK).
4. La anulación (POST /api/ventas/:id/anular) marca la venta como 'ANULADA', reintegra las existencias y genera un movimiento de ingreso con auditoría.

#### 4.6 Módulo 6: Garantías Post-Venta y Primer Service Gratuito (90 días)

**Qué hace:** Garantiza que toda bicicleta nueva adquirida en DN Bike cuente con 90 días de cobertura y derecho a un mantenimiento técnico de calibración sin costo.

**Cómo lo hace:** En `Detalle_Venta`, si el producto vendido posee `tipo_prod = 'bicicleta'`, se habilitan las columnas `primer_service_realizado`, `fecha_primer_service` y `service_excepcion`. El endpoint GET /api/ventas/garantias calcula mediante SQL la diferencia `CURRENT_DATE - fecha_venta`. Si está dentro de los 90 días, el sistema permite registrar la ejecución mediante PUT /api/ventas/primer-service/:id\_detalle vinculando al mecánico responsable.

#### 4.7 Módulo 7: Taller Mecánico, Reparaciones y Tablero Kanban

**Qué hace:** Controla el ciclo de vida de los servicios técnicos mediante un tablero Kanban de 4 estados (Recibida, En Reparación, Lista, Entregada) y gestiona el consumo de repuestos.

**Cómo lo hace:** POST /api/repares crea la orden de trabajo vinculando bicicleta, cliente y mecánico. A través de POST /api/detalles-reparacion se agregan repuestos a la orden: el servicio descuenta automáticamente el inventario en Productos y registra el movimiento de stock. El costo final se calcula sumando el costo de todos los detalles técnicos más la tarifa de `costo_mano_obra` fijada por el técnico.

#### 4.8 Módulo 8: Proveedores y Pagos Comerciales

**Qué hace:** Registra los proveedores mayoristas con CUIT y asienta pagos de compras e insumos para el cálculo del balance neto de la bicicletería.

**Cómo lo hace:** A través de POST /api/pagos-proveedor se registra el egreso con monto, fecha, comprobante y método de pago (cheque, transferencia bancaria o efectivo). Este egreso impacta directamente en las métricas de balance financiero mensual de la empresa.

#### 4.9 Módulo 9: Inteligencia de Negocio, Reportes y Balances

**Qué hace:** Agrega métricas financieras y operativas por rangos de fecha: total vendido, ingresos del taller técnico, egresos comerciales, margen neto de ganancia y capital inmovilizado en bicicletas de stock.

**Cómo lo hace:** GET /api/reportes/balance ejecuta consultas optimizadas con funciones agregadas (SUM, COUNT) filtrando ventas activas (WHERE estado != 'ANULADA'). Los reportes de capital inmovilizado multiplican `SUM(cantidad * precio)` de bicicletas en inventario.

#### 4.10 Módulo 10: Bitácora de Auditoría y RespalDOS Automatizados de Base de Datos

**Qué hace:** Garantiza la trazabilidad inmutable de eventos sensibles del sistema y permite al Administrador descargar un volcado completo de la base de datos SQL para respaldo de seguridad externo.

**Cómo lo hace:** Cada acción crítica dispara una llamada a `BitacoraService.registrar()`, asentando una fila en `Bitacora_Actividad` con fecha, usuario, módulo y descripción. Para el respaldo, el endpoint `GET /api/backup/descargar` genera un archivo SQL estructurado con DDL y datos de todas las tablas en streaming hacia el navegador del cliente.

5

## Conclusiones y Consideraciones de Mantenibilidad

La implementación definitiva de **BikeSystem** consolida con éxito los objetivos formativos y técnicos de la cátedra de *Prácticas Profesionalizantes II*:

- **Coherencia Metodológica:** Trazabilidad completa desde los Requerimientos Funcionales y Diagramas de Casos de Uso hasta el modelo físico PostgreSQL y los componentes de interfaz en React.
- **Robustez Transaccional:** Erradicación total de inconsistencias de stock gracias a la unificación de movimientos en la tabla Movimiento\_Stock y transacciones ACID en ventas, compras y taller.
- **Calidad y Usabilidad:** Interfaces ergonómicas validadas con el usuario final, soporte offline/LAN mediante Electron, y búsqueda masiva instantánea con índices Trigram.

**Validación Final de Entrega Completa del TPI**

Con este documento se da cumplimiento total a los requerimientos de la **Entrega Completa del TPI (06/10/2026)**, incorporando la Primera Entrega Corregida con las 3 observaciones resueltas, la arquitectura de software explicada, los 4 diagramas UML vectoriales legibles y la descripción técnica integral de todos los módulos del software.

| FICHA TÉCNICA Y METADATOS DE ENTREGA FORMAL |                           |                                 |                      | Versión Definitiva 2.0 • Aprobada |  |
|---------------------------------------------|---------------------------|---------------------------------|----------------------|-----------------------------------|--|
| Institución Académica:                      | Cátedra / Año:            | Equipo de Desarrollo:           | Docente Evaluador:   |                                   |  |
| ITEC El Molino                              | Prácticas Prof. II (2026) | Germán Domingues, Exequiel Ríos | Prof. Nicolás Kappes |                                   |  |